Multi-Agent Patterns, Workflows and Orchestration

Dr. Trung Nghia Duong

Gisma University of Applied Sciences

Gisma University of Applied Sciences

Learning objectives

- By the end of this session, students should be able to:
  - Explain the difference between workflow patterns and autonomous orchestration patterns
  - Describe sequential, conditional, parallel, and conversation-driven multi-agent patterns
  - Implement a simple sequential workflow using *FunctionStep* and *Workflow*

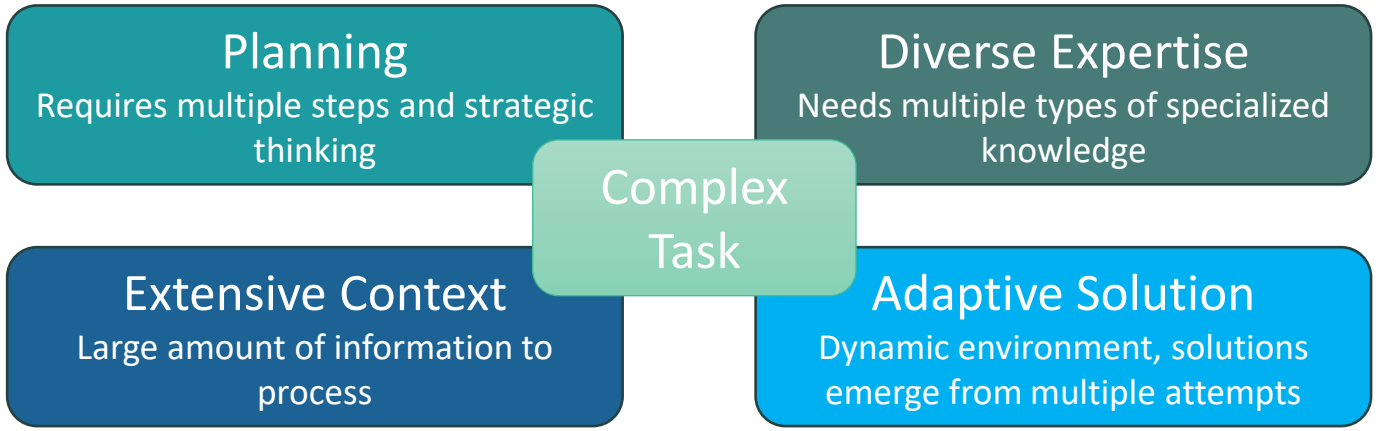
Learning Resources

- Git Repo: https://github.com/duongtrung/teaching/tree/main/tutorials/0011.%20picoagents-tutorials
  - Jupyter notebooks, quiz, media library
- Pre-class preparation
  - Dibia, V. (2025). Chapters 1, 2, 6, 7.
  - Infante, R. (2026). Chapters 1, 2 (excepts MCP section).
  - Read the Appendix and prepare the coding environment.
- Post-class
  - Watch media library

Gisma University of Applied Sciences

Task Complexity Levels

Complexity Level	Task Example	Description
Model-Level	What is the height of the Berliner Fernsehturm?”	Direct information retrieval from training data
Agent-Level	“Tell me the number of students studying at Gisma University until today”	Requires current data, planning, and tool use
Multi-Agent	“Build a machine learning leaderboard that students can submit codes and view participants’ performance”	Involves multiple domains of expertise and iterative development



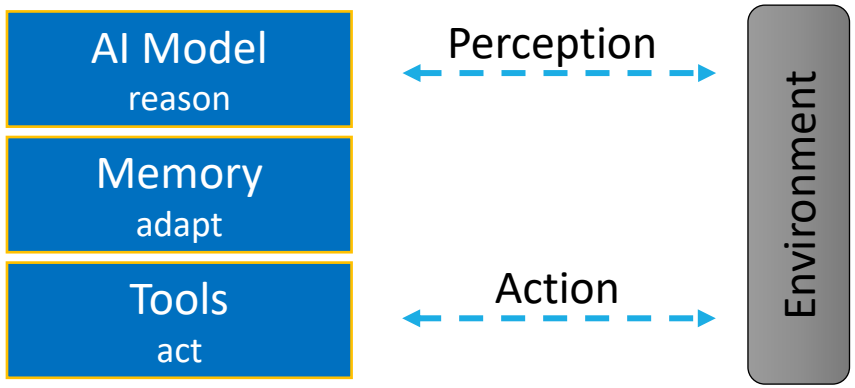
Gisma University of Applied Sciences

Definitions: Agents, Multi-Agent Systems, and Tools

Dibia, V. (2025).

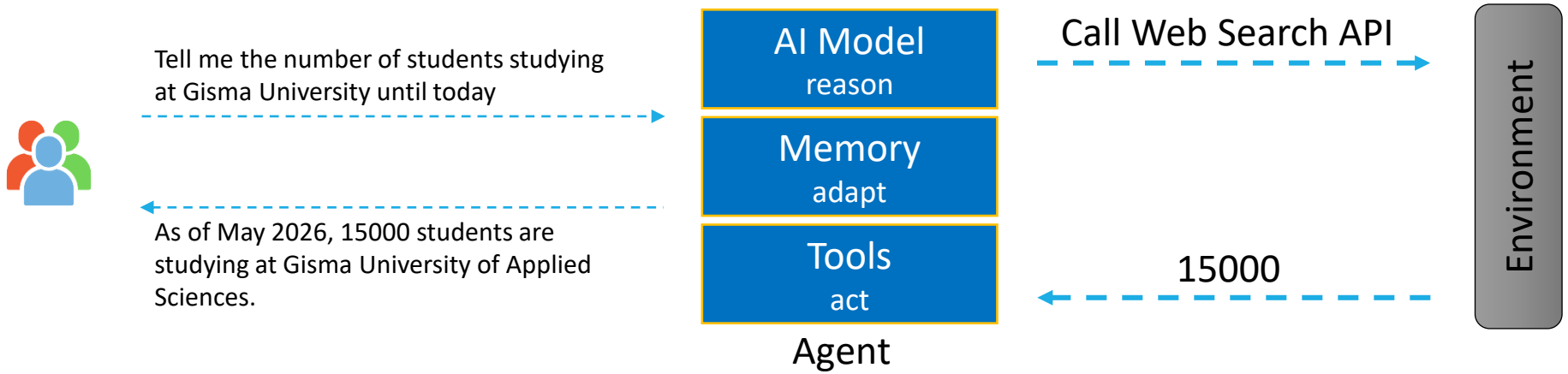
Russell, S., & Norvig, P. (2022).

- Agent: An AI system that can reason, act with tools, communicate, and adapt.
- Multi-Agent System: Multiple agents working together, each with specialized capabilities.
- Tools: External capabilities like APIs, code execution, web search that extend agent abilities.



How Agents Work

- Agents operate through a fundamental **action-perception loop**



The agent action-perception loop: Agents operate through iterative cycles where they take action (such as calling an API), perceive the results (processing the response), and adapt their approach based on outcomes. When successful, agents provide natural language responses; when errors occur, they may retry with different parameters or inform users of limitations.

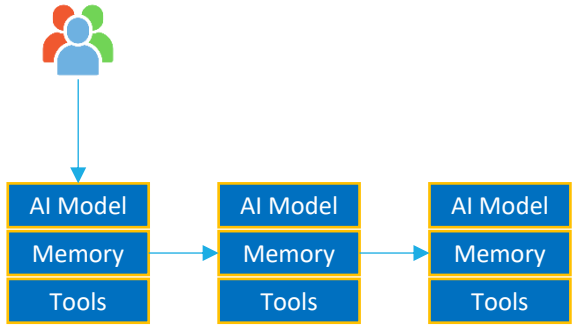
How Agents Work

- Model
  - The reasoning engine that enables decision-making and planning. This is typically a LLM or large multimodal model (LMM) that can understand context, generate plans, and determine appropriate actions.
- Tool
  - Tools, also known as skills or plugins, are specific implementations of logic designed to carry out particular tasks. They serve as the primary method for agents to act on tasks.
    - General-purpose tools enable a broad range of capabilities.
    - Domain-specific tools are designed to address a specific task or a set of related tasks
- Memory
  - Short-Term Memory acts as working memory for the current task, tracking recent actions, conversation history, and temporary information needed to complete the immediate objective.
  - Long-Term Memory stores accumulated knowledge, successful strategies, and learned patterns that persist across different tasks and sessions.

What is a Multi-Agent System?

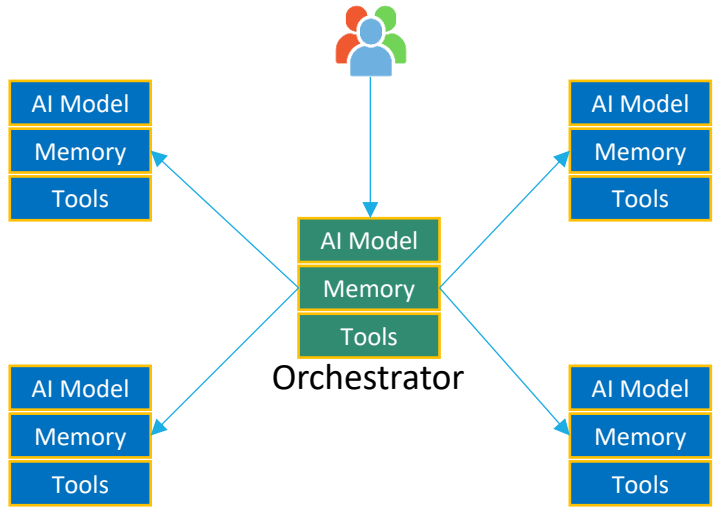
- Applications that involve a group of agents, each with diverse capabilities and specialized objectives, collaborating to solve tasks
- Multi-Agent orchestration workflows

Workflow (graph) orchestration



Control flow is predefined (typically modelled as a graph)

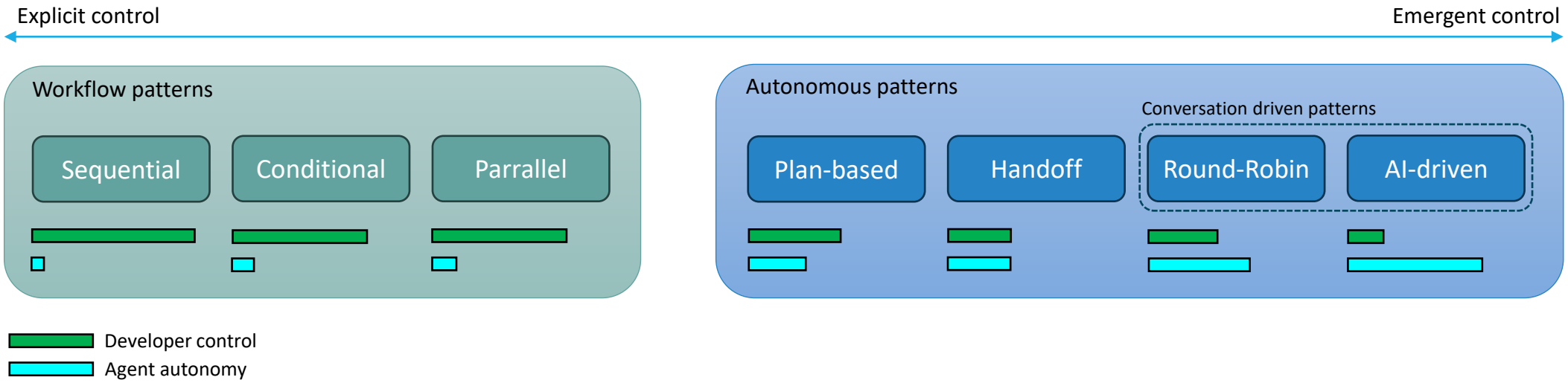
Autonomous (AI Driven) orchestration



Control flow is driven by an AI model at runtime across several patterns

Plan-based | AI turn taking | Handoff

Autonomy Spectrum for Multi-agent Systems



- AI agents introduce fundamental differences:
  - Semantic reasoning using natural language rather than rules/heuristics
  - Dynamic planning that adapts based on intermediate results
  - Probabilistic outputs requiring new error detection approaches, and
  - Token-based economics where communication costs scale with conversation

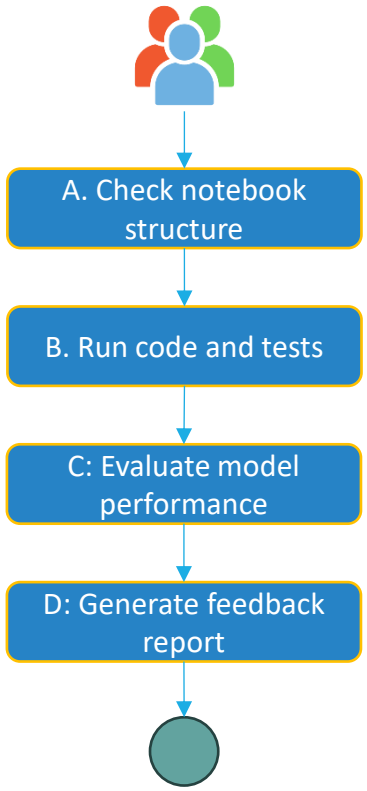
Workflow Patterns (Explicit Control)

- Workflow patterns provide developer-defined execution paths with predictable behavior.
- The system is modeled as a computational graph.
  - Node: a unit of computation
  - Edge: a transition between nodes
  - Condition: logic that decides whether an edge is activated
- The developer already knows the process
  - The solution path must be predefined
  - The task decomposition must be known
  - The system is less adaptive at runtime
  - The developer must encode routing decisions

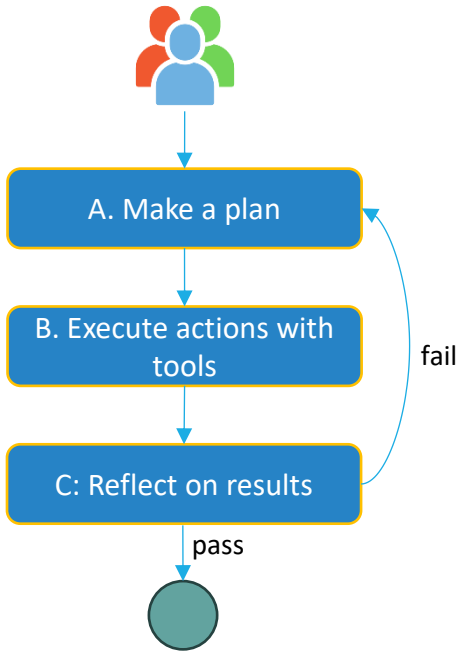
Gisma University of Applied Sciences

Workflow Patterns (Explicit Control)

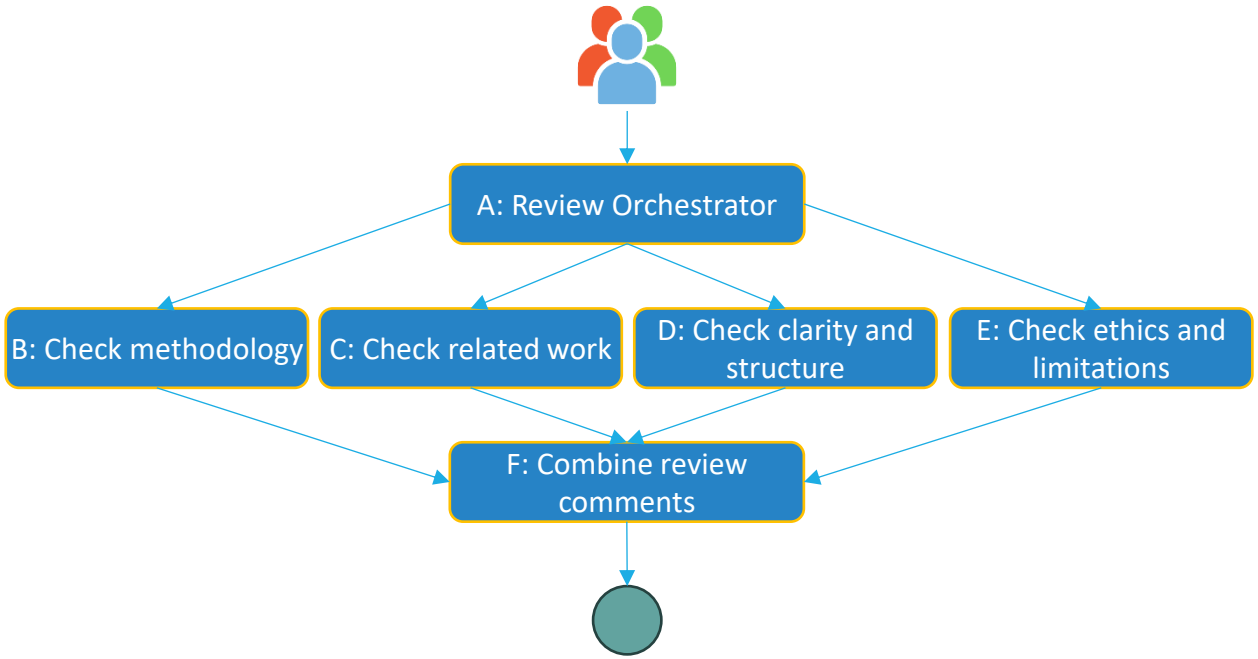
Sequential workflow



Conditional workflow



Parallel workflow



Autonomous Patterns (Emergent Control)

- Autonomous patterns enable **runtime**-determined execution based on task state and agent reasoning.
- A critical idea here is that the **flow of control is driven by an AI model** and hence dynamically determined at runtime.
- Rather than following prescribed paths, agents orchestrate through communication and shared understanding of the current task context.
- An important design principle applies to both workflow and autonomous patterns: any “agent” may itself be a multi-agent system internally.

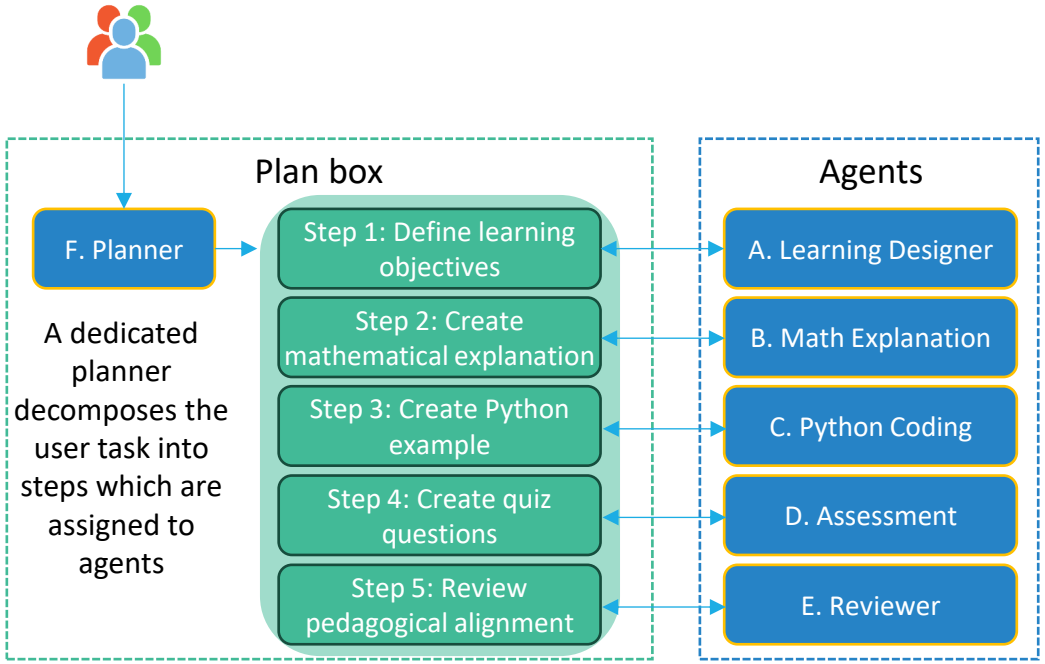
Plan-based Orchestration Pattern

Mialon, G. et al. (2024). ICLR.

Fourney, A. et al. (2024). Microsoft Research.

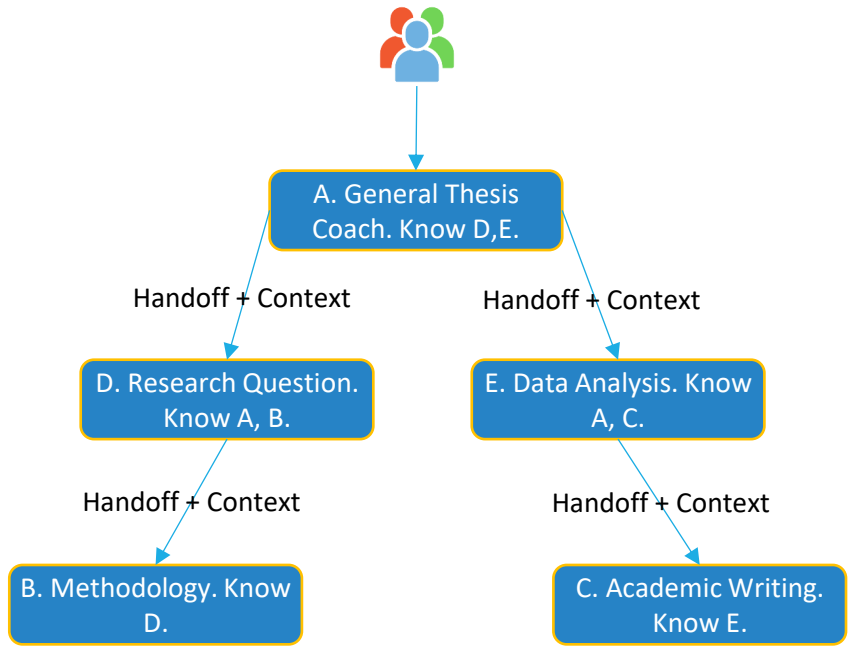
Anthropic Engineering Blog. (2025).

- Employs a single orchestrator agent to manage entire task execution through **autonomous** plan creation, dynamic task assignment, and centralized progress monitoring.
- This agent acts as a “project manager”, creating plans, assigning work, reviewing progress, and orchestrating between specialized agents.
- Control flow characteristics
  - Plan management: Orchestrator maintains explicit task plans with assignments and dependencies.
  - Visibility: Orchestrator sees all context; other agents receive only relevant information.
  - Task assignment: Explicit work distribution by the orchestrator based on the current plan.
  - State management: Centralized in the orchestrator.



Handoff Pattern

- Agents make local decisions about when and to whom they should transfer control based on their understanding of the task and knowledge of other available agents.
- Control flow characteristics
  - Turn-taking: Direct handoff via explicit transfer mechanisms.
  - Visibility: Each agent knows only a subset of other agents and their capabilities.
  - Decision making: Local decisions based on task needs and known agents.
  - State management: Explicitly passed between agents during handoff.

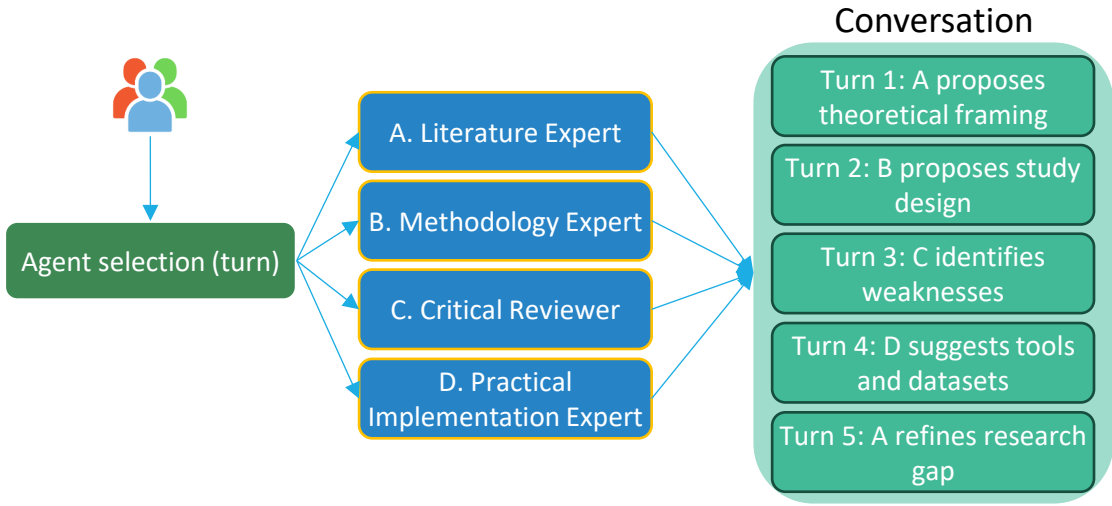


Conversation-driven Pattern (Group Chat)

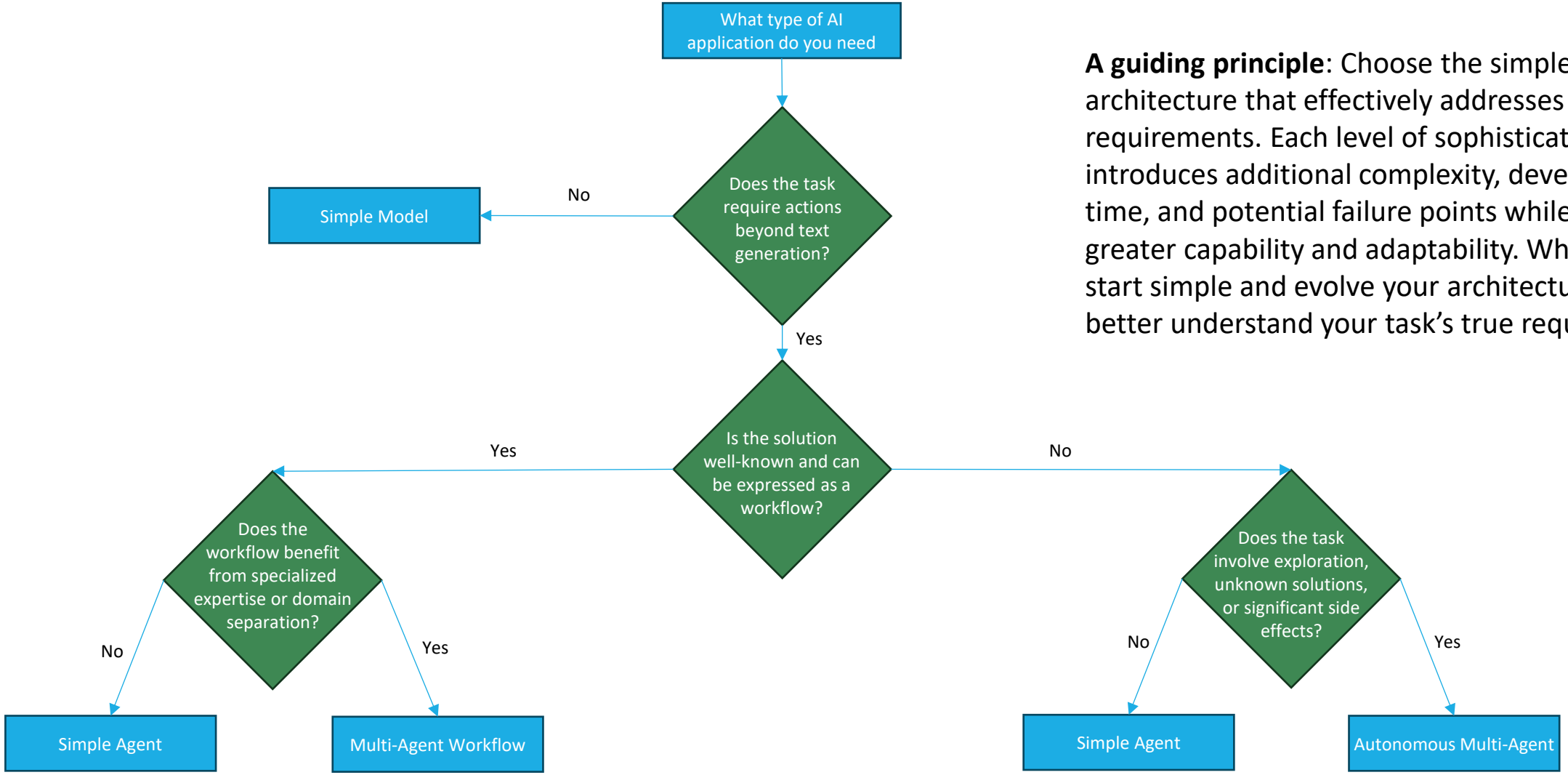
Yao, S. et al. (2023). ICLR.

Shinn, N. et al. (2023). NeurIPS.

- All agents participate in a shared conversation where orchestration emerges through turn taking as part of a dialogue.
- Control flow characteristics
  - Turn-taking: Determined by selection mechanisms (round-robin, random, or AI-driven).
  - Visibility: Broadcast – all agents observe all messages in the shared conversation.
  - Decision making: Next speaker selected based on conversation context, not predetermined plans.
  - State management: Implicit in the conversation history.



Choosing the Right AI Agent Architecture



A guiding principle: Choose the simplest architecture that effectively addresses your requirements. Each level of sophistication introduces additional complexity, development time, and potential failure points while offering greater capability and adaptability. When in doubt, start simple and evolve your architecture as you better understand your task's true requirements.

Choosing the Right AI Agent Architecture

- Rather than asking “which pattern is best?”, ask “what does my task need?”
  - Control flow: how execuation moves between agents.
  - Autonomy: how flexibly the system can explore solution paths not explicitly programmed by developers.
  - Control: how much predictable control developers have over system behavior.
  - Complexity: the implementation and operational complexity required.

coding in 2022

JS stringUtils.js

```
function startsWithCapitalLetter(str) {
  return str.length > 0 && str[0] === str[0].toUpperCase();
}
```

coding in 2027

JS stringUtils.js

```
import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY,
});

export async function startsWithCapitalLetter(str) {
  const prompt = `You are a capitalization expert.
    You have been studying capitalization for years

    Text to analyze: """$str}"""

    Respond with ONLY 'true' or 'false'.
    Is the first letter of the text a capital letter?`;

  const response = await anthropic.messages.create({
    model: 'claude-mythos-1-20261225',
    max_tokens: 5000000,
    temperature: 0,
    thinking: 'ultramaxmegathink',
    messages: [{ role: 'user', content: prompt }],
  });

  const answer = response.content[0].text.trim().toLowerCase();
  return answer === 'true';
}
```

References

- Dibia, V. (2025). *Designing Multi-Agent Systems: Principles, Patterns and Implementation for AI Agents*. Victor Dibia, multiagentbook.com.
  - https://multiagentbook.com/
- Infante, R. (2026). *AI Agents and Applications With LangChain, LangGraph, and MCP*. Manning.
  - https://www.manning.com/books/ai-agents-and-applications
- Microsoft AI Blog. (2024). *AI at Work Is Here. Now Comes the Hard Part*. https://www.microsoft.com/en-us/worklab/work-trend-index/ai-at-work-is-here-now-comes-the-hard-part
- Y Combinator. (2026). *The AI Agent Economy Is Here*. https://www.ycombinator.com/library/NK-the-ai-agent-economy-is-here
- Russell, S., & Norvig, P. (2022). Artificial Intelligence: A Modern Approach, 4th Global ed. *Harlow, UK: Pearson Education Limited*.

References

- Google Blog. (2025). Developer's guide to multi-agent patterns in ADK. https://developers.googleblog.com/developers-guide-to-multi-agent-patterns-in-adk/
- Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., & Scialom, T. (2024). Gaia: a benchmark for general ai assistants. In International Conference on Learning Representations (ICLR).
- Fourney, A., Bansal, G., Mozannar, H., Tan, C., Salinas, E., Niedtner, F., ... & Amershi, S. (2024). Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*.
- Anthropic Engineering Blog. (2025). How we built our multi-agent research system. https://www.anthropic.com/engineering/multi-agent-research-system
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, *36*, 8634-8652.

Hands-on Lab

- Scenario:
  - In the course Introduction to Machine Learning at Gisma University, a student submits an HTML-exported Jupyter notebook for an ML assignment.
  - The review pipeline follows a fixed sequence:
    - Check notebook structure
    - Run basic code checks
    - Evaluate model-performance evidence
    - Generate feedback summary
    - This is a sequential workflow because each step depends on the previous one.
  - Adjust the codes to solve three tasks.
    - Task 1: Add a new deterministic step called check_references between run_basic_code_checks and evaluate_model_evidence.
    - Task 2: Change the scoring rubric so includes_hyperparameter_search contributes 15 points and code_ok contributes 35 points.
    - Task 3: Extend FeedbackOutput with a pass_fail field and populate it based on score >= 70.

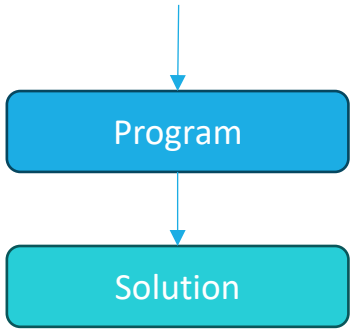
Appendix

- From Software 1.0 (Rules) to Software 3.0+ (Autonomous MultiAgent Systems)
- Environment Setup

From Software 1.0 (Rules) to Software 3.0+ (Autonomous MultiAgent Systems)

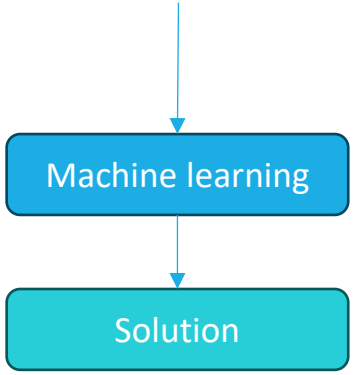
Software 1.0

Developers encode solution as rules and conditions expressed in a programming language.



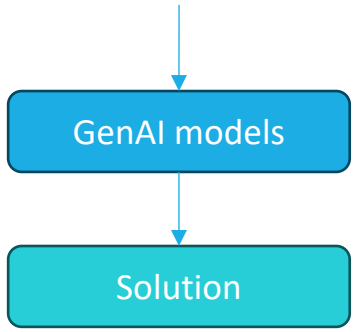
Software 2.0

Machine learning algorithms are used to find solution rules directly from data.

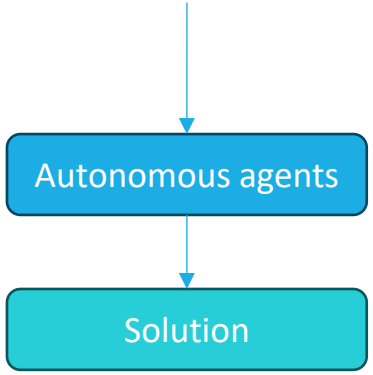


Software 3.0

Natural language instructions and program chains drive capable GenAI models in solving tasks.



Autonomous agents collaborate with other agents and humans to address tasks.



++ increasing: adaptability, developer efficiency

Environment Setup

- Set up the codes
  - https://github.com/victordibia/designing-multiagent-systems
  - There are there options to getting started: interactive notebooks, GitHub codespaces, and local installalation
  - An OPENAI_API_KEY required
  - Run a hello_agent notebook at: https://github.com/duongtrung/teaching/blob/main/tutorials/0011.%20picoagents-tutorials/0001.%20hello_agent.ipynb
- Learning materials and hands-on lab notebooks
  - https://github.com/duongtrung/teaching/tree/main/tutorials/0011.%20picoagents-tutorials